

Did the Stock Airspy Image Actually Need Improvement?

Yes. Not because it was old, and not because a deeper queue is automatically better. It needed improvement because the original firmware contained several paths that could silently discard valid USB packets, overwrite or skip sample banks, spin a core at full speed, or deliberately stop forever when ordinary hardware reported an error.

Baseline	Upstream Airspy firmware 946184a, 2016-09-19
Final image	V5c universal R2/Mini, 21,196 bytes
Final SHA-256	138d64f1a3dd4007f813264f721435729b6180cd5ff0851baa36939586b888d5
Scope	Firmware transport, DMA, memory, lifecycle and recovery; driver changes excluded
Date	24 July 2026

Editorial position. The stock image was clever, compact and usable on an uncontested USB link. That does not make its failure semantics acceptable. The defects below are judged from the code and from the failures we reproduced, not from a claim that every stock radio regularly crashed.

How to read this report

This is not a changelog and it is not a claim of formal hardening. It is a ledger of defects. For each item, the original source is shown, the collision or failure mechanism is stated, and the practical likelihood is estimated.

Probability language is deliberately conditional. We do not have enough fleet data to invent per-hour failure rates. A DMA error may be very rare on healthy hardware, while the stock response to that error is still almost certain to wedge. Conversely, the one-dTD spin happens routinely but usually resolves; its harm is heat and lost scheduling margin rather than an immediate crash.

Rating	Meaning in this report
Very high	Deterministic or greater than roughly 50% once the qualifying event occurs.
High	A likely outcome under the named event or workload; roughly 10-50%.
Moderate	Plausible in real use but dependent on timing, host behavior or a peripheral fault.
Low	A real defect with a narrow or unusual trigger; usually below roughly 1% per ordinary session.
Unknown	The code proves the failure path, but field exposure was not measured.

What is included

Part I contains defects inherited from stock. Part II contains pressure points and collision hazards that became visible when the transport was deepened. Part III records regressions we introduced during redevelopment and caught before the final V5c image. Those are not charged to the stock authors.

What is not claimed

V5c has not completed months-long qualification or systematic fault injection. Several startup peripheral waits and CPU exception handlers remain deliberately outside the claim. The correct conclusion is 'materially better and much less likely to wedge,' not 'formally hardened.'

Executive verdict

The stock firmware did need improvement. Its ordinary success depended on a favorable timing envelope: the M0 had to notice each half-buffer transition quickly, its single bulk descriptor had to retire promptly, and USB/DMA hardware had to avoid exceptional states. When those assumptions failed, the code often had only three responses: spin, silently lose continuity, or stop forever.

The most serious stock findings are not stylistic. They are direct control-flow facts:

Defect	Stock consequence	Practical judgment
GPDMA error clear compiled out	Asserted IRQ can consume M4 indefinitely.	Rare trigger; near-certain wedge if triggered.
USB dTD error executes while(1)	M0 deliberately stops forever.	Rare trigger; deterministic wedge.
Restart resets only device data toggle	Host discards first 512-byte packet after odd stop.	Reproduced; deterministic conditional loss.
Offset-only two-bank handoff	No ownership proof; delay can overwrite or hide an entire wrap.	Moderate under contention; high at limits.
ADC starts before tuner setup	Capture runs while M0 is unavailable to feed USB.	Observed startup FIFO loss at 10 MS/s.
Unbounded hardware/core waits	One failed transition wedges control handling.	Low trigger; high conditional severity.

V5c changes the failure model. It has ten explicit bank owners, ten dTDs, bounded queue operations, correct data-toggle lifecycle, recoverable USB and GPDMA error paths, deadlines for the transport-critical waits, and telemetry that distinguishes ADC loss from USB delay. The important improvement is not merely throughput. It is that failures now have names and bounded consequences.

Part I - Bugs present in the stock image

1. The two-bank offset protocol has no ownership

Original: `airspy_m4/airspy_m4.c` and `airspy_m0/airspy_m0.c`

```
/* M4: publish only a wrapping offset */
set_usb_buffer_offset(
    inc_mask_usb_buffer_offset(get_usb_buffer_offset(),
                               USB_DATA_TRANSFER_SIZE_BYTE));
signal_sev();

/* M0: compare only the current offset */
uint32_t offset = get_usb_buffer_offset();
if (offset != *last_offset_m0) {
    usb_transfer_schedule_block(&usb_endpoint_bulk_in,
                               &usb_bulk_buffer[offset], length);
    *last_offset_m0 = offset;
}
```

Why it is a bug. An offset says where the producer is now; it does not say which completed bank USB owns, whether a descriptor names that bank, or whether one or more transitions were missed. At 10 MS/s, a 16 KiB bank completes in about 409.6 us and the entire two-bank cycle repeats in about 819.2 us. If M0 is delayed across two transitions, the offset returns to its prior value and the consumer sees no change at all. If it is delayed across one transition, DMA may return to a bank still being read by USB.

Evidence. The redevelopment repeatedly exposed startup and contention losses once counters were available. Stock has no counter capable of distinguishing the same events.

What changed. V5c assigns each of ten banks produced, submitted and retired generations. DMA checks the bank two positions ahead and halts before overwrite. A bank is reusable only after USB retirement.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Every stream; harmful only when M0/USB falls behind for roughly 0.4-0.8 ms at 10 MS/s.	Silent gap, stale/mixed payload, or apparent stall; crash is unlikely.	Moderate normally; high on a contested link or during long M0 work.	Direct timing bound and absence of stock ownership state.

2. One bulk dTD turns pressure into a hot spin

Original: `airspy_m0/usb_endpoint.c` and `common/usb_queue.c`

```
static USB_DEFINE_QUEUE(usb_endpoint_bulk_in, 1);

int usb_transfer_schedule_block(...) {
    int ret;
    do {
        ret = usb_transfer_schedule(endpoint, data, maximum_length);
    } while (ret == -1);
    return 0;
}
```

Why it is a bug. When the only dTD is busy, the scheduler retries allocation continuously. In the main loop, interrupts can eventually retire the descriptor, so this is not always a deadlock. It is still an avoidable full-speed polling loop precisely when USB is late. It consumes M0 cycles, raises thermal load and steals margin from control and transport work.

Evidence. The stock structure itself proves routine exposure whenever production reaches the scheduler before the prior 16 KiB transfer retires.

What changed. V5c uses a ten-dTD pool and a nonblocking submission pass. Pool exhaustion records backpressure and returns to the event loop; it never hides policy inside an allocator spin.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Common at high stream rates and whenever USB completion trails capture.	Usually resolves; raises heat and latency. Can amplify another delay into sample loss.	High exposure, moderate direct severity.	The loop is unconditional and bulk depth is exactly one.

3. Capture starts before the M0 finishes tuner setup

Original: `airspy_m0/airspy_m0.c`

```
void ADCHS_start(uint8_t conf_num) {
    start_stop_adchs_m4(conf_num, START_ADCHS_CMD);

    i2c0_init(...);
    i2c1_init(...);
    r820t_init(...);
    r820t_set_if_bandwidth(...);
}
```

Why it is a bug. The first call starts ADC/DMA on M4. Only afterward does M0 reinitialize I2C and perform R820T setup. During that synchronous tuner work, the sample producer is live but the USB producer loop cannot run. At the R2 limit, the buffering window is measured in microseconds.

Evidence. On the R2 at 10 MS/s, initial tests recorded one or two FIFO overflows per finite start. Timed telemetry showed the count was created at startup and remained unchanged through tens of thousands of banks. Moving tuner setup before ADC start eliminated the event in ten repeated starts and an 84,430-bank run.

What changed. The prepared-start protocol raises clocks and completes tuner work while ADC remains stopped, allows host reads to be submitted, and only then starts ADC/DMA.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Every stock start; actual loss depends on sample rate and host timing.	Startup sample discontinuity or FIFO loss; normally no crash.	High at 10 MS/s; moderate at lower rates.	Hardware telemetry and repeated before/after starts.

4. Receiver restart resets only the device's USB data toggle

Original: `airspy_m0/airspy_rx.c`

```
void set_receiver_mode(const receiver_mode_t new_receiver_mode) {
    usb_streaming_disable();
    if (new_receiver_mode == RECEIVER_MODE_RX) {
        usb_endpoint_init(&usb_endpoint_bulk_in);
        ADCHS_start(sample_rate_conf_no);
    }
}
```

Why it is a bug. Endpoint initialization resets the device bulk-IN toggle to DATA0. An application-level stop/start is not a USB reset, SET_CONFIGURATION, SET_INTERFACE or CLEAR_FEATURE(HALT), so the host preserves its own toggle. If the prior stream ended after an odd packet count, host and device disagree. The host ACKs the first new packet as a duplicate and discards it; the device reports success.

Evidence. The hardware fixture lost exactly 512 bytes on five of five qualifying restarts: 65,024 of 65,536 bytes arrived while all dTDs reported success. Odd/even cycles alternated failure and success. After separating init from resume, ten odd-packet restarts and a 4,404-dTD stress run passed.

What changed. V5c uses endpoint initialization only at USB configuration boundaries. Application resume preserves the shared DATA0/DATA1 state.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
About half of arbitrary restarts qualify if stop packet parity is effectively random.	Deterministic 512-byte silent loss on a qualifying restart; no crash.	Very high conditional loss, moderate per restart.	Exact odd/even reproducer on the Mini.

5. Stop and reset revoke USB before stopping the producer

Original: `airspy_m0/airspy_rx.c` and `common/usb.c`

```
void set_receiver_mode(...) {
    usb_streaming_disable(); /* transport first */
    if (new_receiver_mode != RECEIVER_MODE_RX)
        ADCHS_stop(sample_rate_conf_no); /* producer later */
}

static void usb_bus_reset(...) {
    usb_reset_all_endpoints(); /* queues reset first */
    usb_set_address_immediate(device, 0);
    usb_set_configuration(device, 0); /* callback stops receiver later */
}
```

Why it is a bug. Both sequences briefly permit DMA to produce into buffers after transport ownership has been disabled or flushed. That is a classic producer/consumer teardown race. A completion or reset inside the window can free a descriptor while DMA or software still relies on its buffer.

Evidence. The timing window is narrow and stock has no ownership telemetry, so field incidence cannot be measured from the original image.

What changed. All active-state changes now stop the ADC producer before cancelling USB ownership. Bus reset notifies the application and stops capture before endpoint queues are reconstructed.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Stop, sample-rate change, packing change and USB reset.	Usually hidden at teardown; can create stale completion, false-free ownership or restart wedge.	Low to moderate per lifecycle event.	Definite ordering violation; trigger requires an unlucky completion window.

6. Production GPDMA errors - and ADC fault flags - are never acknowledged

Original: `airspy_m4/airspy_m4.c` (`DMA_ISR_DEBUG` is disabled)

```
#undef DMA_ISR_DEBUG

void dma_isr(void) {
#ifdef DMA_ISR_DEBUG
    status = LPC_ADCHS->STATUS0;
    LPC_ADCHS->CLR_STAT0 = status;

    status = LPC_GPDMA->INTERRSTAT;
    if (status) {
        stat_adchs.dma_err_cnt++;
        LPC_GPDMA->INTERRCLR |= status;
    }
#endif
    status = LPC_GPDMA->INTTCSTAT;
    ...
}
```

Why it is a bug. The only read and clear of the DMA error status is compiled out of the normal image. A level-like error interrupt can therefore remain asserted and immediately re-enter `dma_isr` forever. M4 loses the ability to advance capture or service commands. The ADC STATUS0 read/clear is trapped behind the same debug guard, so production also leaves FIFO, descriptor and range fault flags latched and unobservable. Clearing DMA and blindly continuing would still be wrong because the current bank may be incomplete.

Evidence. Source provenance confirms this defect exists in vanilla, V3 and the live-qualified V4; it was not introduced by the queue rewrite.

What changed. V5c always clears error status, halts channel 0, invalidates the uncertain epoch, asks M0 to cancel USB owners, verifies every bank is free, rebuilds ADC/DMA, and resumes with a counted gap.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Rare hardware/DMA fault; no fleet rate is available.	Near-certain M4 interrupt storm or permanent capture wedge in stock.	Low trigger probability, critical conditional severity.	Direct preprocessor/control-flow proof.

7. USB errors and queue invariant failures deliberately stop M0 forever

Original: `common/usb_queue.c`

```
if (endpoint_queues[index] == NULL) while (1);
if (endpoint_queues[index] != NULL) while (1);

if (status & USB_TD_DTD_TOKEN_STATUS_HALTED
    || status & USB_TD_DTD_TOKEN_STATUS_BUFFER_ERROR
    || status & USB_TD_DTD_TOKEN_STATUS_TRANSACTION_ERROR)
{
    // TODO: Uh oh, do something useful here
    while (1);
}
```

Why it is a bug. This is not a missing optimization. It is a production infinite loop in the completion path. Any dTD that retires with HALTED, BUFFER_ERROR or TRANSACTION_ERROR causes M0 to stop handling USB, control requests and stream ownership permanently until external reset. Queue lookup and duplicate-registration invariant failures use the same stop-forever policy.

Evidence. The error trigger is uncommon on a healthy short cable, but USB errors become more plausible with marginal power, hubs, disconnects, electrical noise and aggressive contention.

What changed. V5c retires the failed descriptor with status, cancels later descriptors stranded behind the halted queue, clears queue-head halt state, re-primers, and continues. A short transfer is counted as a discontinuity rather than ignored.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Rare USB transaction, buffer or endpoint-halt error.	Deterministic M0 wedge in stock.	Low trigger probability, critical conditional severity.	Explicit while(1) on the hardware error bits.

8. EP0 can allocate the next stage before freeing the completed one

Original: `common/usb_request.c` plus blocking scheduler

```
void usb_control_in_complete(usb_endpoint_t* const endpoint) {
    if (device_to_host)
        usb_request(endpoint, USB_TRANSFER_STAGE_DATA);
    else
        usb_request(endpoint, USB_TRANSFER_STAGE_STATUS);

    usb_queue_transfer_complete(endpoint); /* free happens last */
}
```

Why it is a bug. The request handler can schedule the next control stage before the completed EP0 dTD is returned to its four-entry pool. The scheduler it calls is the infinite retry form. If the pool is exhausted inside the USB ISR, the completion processing needed to free a dTD cannot run until the ISR exits: a self-deadlock.

Evidence. Ordinary Airspy requests are short, so this is a narrow trigger. Malformed, overlapping or aborted control transactions make it more credible.

What changed. V5c treats a new SETUP as an abort boundary, retires both old EP0 directions first, clears stale completion flags, frees a normal completion before scheduling the next stage, and stalls EP0 on allocation/validation failure so the next SETUP can recover.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Unusual EP0 overlap, abort or pool exhaustion.	High chance of permanent USB-control wedge once exhausted.	Low in ordinary SDR use; moderate under hostile or faulty control traffic.	Allocation/free ordering plus blocking loop in the same ISR domain.

9. Critical USB hardware transitions have no deadline

Original: `common/usb.c`

```
while (USB0_ENDPTPRIME & mask);
while (USB0_ENDPTFLUSH & mask);
while (usb_endpoint_is_ready(endpoint));
while (usb_controller_is_reseting());

do {
    USB0_USBCMD_D |= USB0_USBCMD_D_ATDTW;
    done = usb_endpoint_is_ready(endpoint);
} while (!(USB0_USBCMD_D & USB0_USBCMD_D_ATDTW));
```

Why it is a bug. Each loop assumes the controller will eventually change state. If an endpoint, PHY, reset or tripwire latch does not progress, M0 remains in the loop forever. Several calls occur inside USB interrupt handling, so the firmware cannot process the event that might otherwise recover it.

Evidence. No stuck-controller event was deliberately injected. The defect is structural and its trigger rate is unknown.

What changed. V5c gives transport-critical waits an approximately 39 ms deadline at 204 MHz or 67 ms at 120 MHz. Failure enters one coherent reset path after producer/ownership safety is established.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Rare controller/PHY fault or lost state transition.	Permanent M0 wedge without an external reset.	Low trigger probability, high conditional severity.	Five explicit unbounded MMIO waits.

10. Cross-core commands can be torn and can wait forever

Original: `airspy_m0/airspy_m0.c`

```
start_adchs->conf = conf_num;
start_adchs->cmd = command;
signal_sev();

while (1) {
    if (start_adchs->raw == 0)
        break;
}
```

Why it is a bug. Command and argument are published as separate byte stores with no single transaction boundary. M4 can theoretically pair a new command with an older configuration if it observes the mailbox between stores. More importantly, the acknowledgement loop has no deadline: a lost event, M4 fault, clock failure or stuck peripheral wedges M0 and therefore USB control indefinitely.

Evidence. The torn-store race is timing-sensitive and probably rare. The no-deadline behavior is certain once M4 fails to acknowledge.

What changed. V5c publishes command and configuration in one aligned 32-bit store with barriers, reads one M4 snapshot, and imposes the same bounded SysTick deadline. Failure causes a whole-MCU core reset rather than unknown cross-core ownership.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Every control command; harmful only on a tiny race or missing acknowledgement.	Wrong configuration, or permanent control-plane wedge.	Low trigger probability; high conditional severity.	Two-store mailbox and explicit infinite acknowledgement loop.

11. USB suspend and controller errors are compiled into comments

Original: `common/usb.c`

```
/*
if (status & USB0_USBSTS_D_SLI) {
    // Device controller suspend.
}
...
if (status & USB0_USBSTS_D_UEI) {
    // USB error.
}
*/
```

Why it is a bug. The controller reports suspend and USB errors, but stock applies no stream policy and records no evidence. During suspend the ADC and tuner remain active while USB ceases consumption. With only two banks, capture rapidly loses continuity. Ignoring UEI also prevents diagnosis and coordinated queue recovery.

Evidence. Suspend is a normal laptop/hub lifecycle event, not a hypothetical hardware failure. Stock gives the user no way to tell whether a later audible catch came from ADC, USB or host software.

What changed. V5c preserves queued ownership across suspend, halts before overwrite, resumes in order, and counts suspend-caused discontinuity. UEI, bus reset and port-change events are counted; dTD errors drive recovery.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Occasional on sleep, hub power management or cable events.	Very likely sample discontinuity; wedge depends on accompanying endpoint state.	Moderate lifecycle risk, low crash risk.	Interrupt branches are disabled and buffering is sub-millisecond.

12. dTD input is trusted beyond the descriptor's legal window

Original: `common/usb_queue.c` and `airspy_m0/airspy_usb_req.c`

```
td->total_bytes = USB_TD_DTD_TOKEN_TOTAL_BYTES(maximum_length) | ...;
td->buffer_pointer_page[0] = (uint32_t)data;
...

usb_transfer_schedule_block(endpoint->in,
    &usb_descriptor_CompatIDDescriptor, endpoint->setup.length);
usb_transfer_schedule_block(endpoint->in,
    &usb_descriptor_ExtProps, endpoint->setup.length);
```

Why it is a bug. The scheduler accepts null/non-null mismatches, lengths beyond the dTD's 15-bit field, and ranges that exceed the five-page address window. The Microsoft descriptor handlers trust the host's wLength instead of clamping to the actual 40-byte and 142-byte objects. An oversized request can make USB DMA read adjacent firmware memory.

Evidence. Normal drivers request the expected lengths, so ordinary exposure is low. A malformed host request or fuzzing makes the overread direct.

What changed. V5c validates pointer, field width and five-page reach before scheduling. The two fixed objects are clamped to their real sizes.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Malformed or oversized control request.	Memory disclosure, invalid DMA read, control stall or device fault.	Low in ordinary use; high under malformed traffic.	Host length is passed directly as DMA length.

13. Queue critical sections restore the wrong interrupt state

Original: `common/usb_queue.c`

```
cm_disable_interrupts();
transfer = queue->free_transfers;
queue->free_transfers = transfer->next;
cm_enable_interrupts();
```

Why it is a bug. The code unconditionally enables interrupts after mutation. If called while interrupts were already disabled by an outer critical section, it prematurely opens that outer section. Nested queue operations can therefore admit an interrupt into state the caller still considers atomic.

Evidence. The stock one-dTD bulk path limits interleavings, but EP0 and later deep queues make nested use realistic. This is a classic low-frequency race rather than a routine failure.

What changed. V5c saves PRIMASK and restores the prior state. The active tail and free-list operations are constant-time, reducing the duration of the correctly bounded critical region.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Nested queue call while an outer caller has masked interrupts.	List corruption, duplicate/free-lost dTD or inconsistent endpoint queue.	Low, but potentially severe.	Unconditional cpsie semantics are visible in source and generated assembly.

14. Write-one-to-clear registers are read-modify-written

Original: `airspy_m4/airspy_m4.c`

```
LPC_GPDMA->INTERRCLR |= status;
...
LPC_GPDMA->INTTCLEAR |= INTTC0;
```

Why it is a bug. A write-one-to-clear register should be assigned the bits being acknowledged. Read-modify-write adds a pointless MMIO read and can clear a status that becomes visible between the read and write. It also obscures the intended hardware semantics.

Evidence. No lost interrupt was reproduced. The window is small, but the path runs at every DMA terminal count and the generated code did perform ldr/orr/str.

What changed. V5c uses direct assignment for the GPDMA clear registers.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Coincident/new status during the read-modify-write window.	Lost accounting or missed DMA event; crash unlikely by itself.	Very low, but hot-path and correctness-relevant.	Source and generated assembly agree on RMW.

15. Unbounded version construction and fixed-object type mismatch

Original: `airspy_m0/airspy_usb_req.c` and `common/airspy_core.c`

```
strcpy((char *)spiflash_buffer,
       (char *)airspy_conf->conf_hw.version);
version_string_len = strlen((char *)spiflash_buffer);
strcpy((char *)&spiflash_buffer[version_string_len],
       version_string);

typedef struct { airspy_conf_t airspy_conf; } airspy_mini_conf_t;
extern airspy_mini_conf_t airspy_mini_conf;
/* Definition has a larger concrete configuration type. */
```

Why it is a bug. The version reply trusts two source strings to fit a 256-byte shared buffer. The hardware configuration objects were also declared through a smaller incompatible external type than their definitions, which caused link-time optimization type-mismatch warnings and created a legal basis for misoptimization.

Evidence. Shipped strings are short, so overflow is unlikely without corrupted configuration. The type mismatch did not produce an observed field failure, but modern LTO correctly warned about it.

What changed. V5c uses bounded copies with explicit source/destination limits and typed accessors returning the common configuration prefix.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Corrupt/long configuration string, or compiler exploiting incompatible declarations.	Buffer overwrite or wrong configuration access.	Very low in normal use; real latent undefined behavior.	Unbounded library calls and compiler LTO diagnostic.

16. The build could silently embed a stale M0 image

Original build relationship: outer ROM image uses `.incbin` without generated dependency

```
/* Conceptual dependency in the M4 ROM image */
.incbin "../airspy_m0/airspy_m0.bin"

/* The compiler-generated dependency file did not name that binary. */
```

Why it is a bug. An incremental M0 rebuild did not necessarily cause the outer M4 image to be relinked with the new payload. A developer could inspect corrected M0 source, run an apparently successful build, and flash an image containing the prior M0 binary. That is a release-integrity bug rather than a runtime algorithm bug.

Evidence. The issue was observed during the hardening pass when an incremental build failed to re-embed the changed M0 payload.

What changed. The ROM-image makefile now lists both embedded M0 binaries as explicit prerequisites. Release instructions still require a clean build.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Incremental firmware development/build.	Flash contains stale transport code while hash/version may appear newly built.	Moderate developer risk; no effect after a verified clean build.	Reproduced dependency omission.

Part II - Collision hazards exposed by a real queue

Stock avoided several classes of queue collision by having almost no queue. That was not free: it converted ordinary latency into spinning and a two-bank overwrite hazard. Deepening the queue forced the firmware to confront hardware races, SRAM topology and descriptor lifetime explicitly.

17. Append has an endpoint-active versus endpoint-idle race

Original append core: common/usb.c

```
tail_td->next_dtd_pointer = new_td;
if (usb_endpoint_is_priming(endpoint))
    return;

do {
    USB0_USBCMD_D |= USB0_USBCMD_D_ATDTW;
    done = usb_endpoint_is_ready(endpoint);
} while (!(USB0_USBCMD_D & USB0_USBCMD_D_ATDTW));

USB0_USBCMD_D &= ~USB0_USBCMD_D_ATDTW;
if (!done)
    usb_endpoint_prime(endpoint, new_td);
```

Why it is a bug. Software can still hold the old tail after hardware consumed it. Appending only the next pointer does not wake an endpoint that has already become idle. The ATDTW tripwire is the required Chipidea decision point: sample endpoint activity atomically, then either leave the appended chain running or prime the new descriptor. A deeper queue makes both branches routine.

Evidence. The Mini fixture deliberately forced 3,000 endpoint-active exits and 101 endpoint-idle/re-prime exits. All 4,404 dTDs retired with exact payloads. This proved both sides of the collision rather than assuming the append branch was unreachable.

What changed. V5c retains the correct tripwire algorithm but makes it bounded, uses an explicit active tail, publishes links with barriers, and has recovery if controller state fails to progress.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Routine whenever appending to a live queue; idle branch depends on precise consumption timing.	Without re-prime, queued data remains stranded and streaming stalls.	High in a deep queue if implemented incompletely.	Forced hardware interleavings demonstrated both branches.

18. Descriptor depth initially collided with the M0 linker region

Observed linker layout when bulk-IN pool changed from one to four

```
/* Stock */
static USB_DEFINE_QUEUE(usb_endpoint_bulk_in, 1);

/* Naive experiment */
static USB_DEFINE_QUEUE(usb_endpoint_bulk_in, 4);
/* Result: M0 16 KiB code/data region overflowed by 768 bytes. */
```

Why it is a bug. The LPC4370 had usable SRAM, but not necessarily in the region the M0 linker was already filling. Increasing a macro did not establish DMA visibility, alignment, non-overlap or owner. The first four-dTD build exceeded the M0 execution/data window and could not be treated as a safe firmware.

Evidence. The linker reported a 768-byte overflow. Separate tests then proved USB DMA visibility for local SRAM1 and SRAM2 before those regions were used.

What changed. V5c places the 64-byte-aligned dTD pool in a bounded metadata section at 0x1001c000 and uses linker assertions. The 2 KiB-aligned queue-head array occupies a separate AHB1 window at 0x20003800.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Any naive queue-depth increase.	Build failure if caught; memory corruption or DMA fault if forced around the linker.	Certain for the tested naive four-dTD layout.	Exact 768-byte linker overflow.

19. USB ownership and DMA ownership can collide on the same bank

Stock has no corresponding check; offending condition expressed by the new contract

```
/* Unsafe condition the stock offset protocol cannot represent */
record->produced_generation != record->retired_generation
/* DMA is about to reuse a bank that USB has not retired. */
```

Why it is a bug. A deeper payload ring is safe only if every dTD names exactly one generation of one bank and no bank returns to DMA until that descriptor retires. Cancellation, short transfer and stale callback paths must release ownership exactly once. Otherwise more buffering merely makes corruption less frequent and harder to diagnose.

Evidence. Synthetic ownership tests and live telemetry exposed ownership halts without out-of-order samples. The final design intentionally drops a bounded gap rather than overwrite an owned bank.

What changed. V5c publishes generation last with a memory barrier, tags every dTD, validates the callback generation, and halts two banks ahead. Recovery refuses to continue if ownership cannot be proven.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Only when USB remains behind long enough to consume the ten-bank reserve or a callback is stale.	Silent mixed/out-of-order samples if unchecked; bounded discontinuity in V5c.	Moderate under severe contention; critical data-integrity consequence.	Explicit ownership invariants and observed halt counters.

20. AHB/SRAM destination ordering caused a deterministic ADC FIFO collision

Intermediate V5 ring order; reconstructed from the measured ten-bank sequence

```
/* Ten banks used five Local SRAM1 destinations and five other slaves.
   The final two entries were both on Local SRAM1. */
... -> LOCAL_SRAM1 -> LOCAL_SRAM1 -> wrap
```

Why it is a bug. The ring had enough aggregate memory and no USB backpressure, yet consecutive GPDMA destinations on the same local SRAM slave created a repeatable service conflict. At the ring boundary, the ADC FIFO flagged almost exactly once per ten-bank revolution. Moving queue heads elsewhere did not change the ratio, isolating destination ordering rather than descriptor storage.

Evidence. The first live V5 run recorded 12,872 FIFO flags in 128,727 banks, nearly 1:10, while USB errors, partials, backpressure and ownership halts remained zero. Strict Local SRAM1/other-slave alternation removed the deterministic event.

What changed. V5c reorders the same ten addresses; it does not add memory or packing. Every Local SRAM1 bank is followed by a destination on another SRAM slave.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Every ten-bank revolution in the offending V5/V5b order.	Deterministic ADC FIFO loss, not a crash.	Certain in the intermediate layout; not a stock defect.	12,872/128,727 live ratio and zero transport counters.

Part III - Regressions we introduced and then removed

A credible audit has to own its own mistakes. The following defects were created while making the old design legible and deeper. They are included because they sharpen the engineering lesson: descriptor and ownership optimization is dangerous when a field that looks immutable is actually controller-owned.

21. V4 cached a dTD pointer that the controller mutates

Offending first-V4 optimization pattern (reconstructed from the corrected source and test record)

```
/* Wrong: cache all five page pointers once, then reuse the dTD. */
if (!transfer->template_initialized) {
    td->buffer_pointer_page[0] = (uint32_t)data;
    td->buffer_pointer_page[1] = ...;
    ...
}

/* Hardware advances page[0] while transferring. */
```

Why it is a bug. Chipidea advances dTD buffer pointer zero as bytes are consumed. Treating page[0] as immutable means a descriptor reused after pool wrap begins from the stale advanced address. Length and ownership counts can still balance, so bookkeeping falsely reports success while payload is wrong.

Evidence. A normal-firmware SPI flash read diverged around byte 1025 and later reads contained zero-filled regions. The independent recovery loader and repeated varying-length reads isolated descriptor reuse. After correction, three readback passes and sustained 10 MS/s streaming matched.

What changed. V5c restores page[0] from the submitted buffer every prepare and caches only immutable pages 1-4.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Every reused descriptor after the first pool revolution in the broken V4 build.	Silent payload corruption; crash unlikely.	Very high and repeatable in that intermediate build.	Exact flash-read divergence and pool-wrap correlation.

22. A stale completion callback could falsely free a newer bank

Offending pre-hardening retirement logic, expressed in its essential form

```
/* Wrong if callback generation is stale */
record->retired_generation = callback_generation;

/* The current submitted_generation may already name a newer owner. */
```

Why it is a bug. USB completion callbacks can arrive after cancellation, restart or queue recovery. Writing an old generation into a record associated with a newer submission can make equality tests claim the bank is free, allowing DMA to overwrite memory still owned by USB.

Evidence. This was identified by static ownership audit rather than a live corruption. The risk appears only because the new design has explicit generations and asynchronous cancellation.

What changed. V5c compares callback generation with the current submitted generation. A mismatch records an ownership fault and leaves the bank owned; the ring halts rather than create a false-free bank.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Stale callback after cancellation/restart/recovery.	Potential silent overwrite and out-of-order samples.	Low trigger probability, critical conditional consequence.	Generation-state proof; no fault-injection qualification yet.

23. A failed dTD could strand every later owner in the hardware chain

Pre-hardening deep-queue behavior

```
/* Hardware halts on the failed dTD.
   Software retires only the head and leaves later ACTIVE records linked. */
FAILED -> ACTIVE -> ACTIVE -> ACTIVE
```

Why it is a bug. On ChipIdea, HALTED/BUFFER_ERROR/TRANSACTION_ERROR on the head can stop advancement through the remaining chain. If software releases only the failed descriptor, every later buffer remains owned by a transfer hardware will never execute. The ADC ring eventually halts permanently.

Evidence. Static queue audit identified the stranded-tail condition. The original stock response was even worse - an immediate while(1) - but a recoverable deep queue needed exact cancellation semantics.

What changed. V5c retires the failed dTD with status, cancels all later descriptors from the halted chain exactly once, clears the queue-head halt, and lets the next submission prime a fresh queue.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Rare USB dTD hardware error.	Permanent ownership leak and stream halt without tail cancellation.	Low trigger probability, high conditional severity.	Hardware halt semantics plus linked owner state.

24. Telemetry reset created a cross-core write race

Pre-V5 hardening behavior

```
/* Wrong ownership model */
M4 start path clears counters that M0 increments for USB events.
```

Why it is a bug. A diagnostic counter still needs a single writer. Clearing M0-owned USB counters from the M4 start path can race an increment, lose evidence or publish a partially inconsistent snapshot. It also erases the event that may explain why the host restarted the radio.

Evidence. The problem was found during the longevity audit. It affects diagnosis rather than sample ownership.

What changed. V5c counters live from firmware boot to firmware reset. The Mac window establishes a new display baseline on start/stop/rate change without writing device counters.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Start coincident with an M0 event increment.	Lost/misleading telemetry; no direct stream crash.	Low behavioral severity, moderate diagnostic importance.	Two-core writer violation.

25. Backpressure loss was initially over-counted

Intermediate V5 counter logic

```
if (adc_status & STAT0_FIFO_OVERFLOW) {
    adc_fifo_overflow_count++;
    backpressure_discontinuity_count++; /* too broad */
}
```

Why it is a bug. An ADC FIFO flag does not by itself prove USB backpressure. It can arise from startup, SRAM arbitration or another capture-side delay. Incrementing the backpressure-loss counter for every FIFO event falsely assigns causation and makes the new observability interface lie.

Evidence. The deterministic one-per-ring SRAM event increased both counters despite zero USB backpressure.

What changed. V5c increments the backpressure-discontinuity counter only when FIFO overflow is observed while resuming from an actual ownership/backpressure halt. Generic FIFO remains a separate counter.

TRIGGER	IF TRIGGERED	PRACTICAL RISK	BASIS
Any ADC FIFO event not caused by ownership halt.	Misdiagnosis only; no sample-path effect.	High occurrence in V5/V5b, low operational severity.	Counter correlation contradicted zero USB/ownership pressure.

What V5c changes about real failure outcomes

The final comparison is best made by outcome, not by feature count. The table below estimates what an operator is likely to observe when the same class of event reaches stock and V5c.

Event	Stock image	V5c	Confidence
Ordinary USB latency shorter than ten banks	One-dTD spin; two-bank overwrite/skip risk.	Absorbed by queue; no loss claim.	High by design and live counters.
USB remains behind beyond ring capacity	No explicit ownership; possible silent corruption/gap.	DMA halts before overwrite; visible discontinuity if FIFO loses samples.	High by state inspection; fault injection incomplete.
Odd-packet application restart	First 512-byte packet discarded.	Toggle preserved; no packet loss in reproducer.	Very high; reproduced.
GPDMA channel-0 error	Uncleared IRQ can wedge M4.	Cancel uncertain epoch, rebuild ADC/DMA, resume with counted gap.	Stock proof high; V5c recovery static/compile checked, not injected.
USB dTD error	M0 executes while(1).	Retire/cancel/re-prime; gap remains observable.	Stock proof high; V5c injection pending.
Stuck USB prime/flush/reset/tripwire	Infinite wait.	Bounded deadline and whole-device recovery path.	Static proof; hardware timeout injection pending.
USB suspend	Ignored; likely silent continuity loss.	Ownership preserved, halt before overwrite, count proven discontinuity.	Moderate; policy implemented, long suspend tests limited.
Unprovable ownership	Not represented.	Remain halted or reset; never guess a bank is free.	High by invariant inspection.

Rough overall likelihood assessment

Stock crashing or hard-wedging during an ordinary short session: probably low on healthy hardware and a clean direct USB link. The radio's field reputation is evidence that the rare error branches are not hit constantly. A reasonable qualitative estimate is well below one percent per ordinary session, but there is no dataset that justifies a narrower number.

Stock losing continuity without crashing: materially higher. Application restart had a deterministic parity-dependent 512-byte loss. Startup work was observed to create FIFO loss at 10 MS/s. The two-bank handoff has under one millisecond of ambiguity at that rate. Under repeated starts, hub contention or a busy host, discontinuities should be considered plausible rather than exceptional.

Stock wedging over days or months: impossible to estimate numerically, but risk accumulates because several rare events have a 100% or near-100% conditional wedge response. The longer the run and the more suspend/reset/cable/peripheral events it encounters, the less reassuring a low per-event trigger rate becomes.

V5c wedging: substantially less likely for the classes addressed here. USB and DMA faults now have bounded recovery, queue pressure is not a spin, and unprovable ownership fails safe. The remaining long-run risk is dominated by unbounded ADC/PLL/I2C/flash startup waits, CPU fault handlers that intentionally stop, and recovery paths not yet fault-injected on hardware.

What remains outside the claim

This report supports calling V5c a materially hardened experimental image. It does not support calling it formally hardened, fail-safe in every peripheral state, or qualified for unattended months-long service.

Remaining exposure	Why it still matters
PLL, ADC reset/FIFO-empty, I2C and flash busy waits	Several startup/reconfiguration loops still lack device-specific deadlines and propagated error results.
CPU exception handlers spin for debugging	HardFault, MemManage, BusFault and UsageFault do not persist a reason and autonomously reboot.
Last-resort reset cause is not persistent	After recovery, the host may know a reset occurred but not the exact pre-reset invariant failure.
Recovery fault injection remains incomplete	The source paths compile and are coherent by inspection, but GPDMA/dTD/controller failures have not all been induced on hardware.
Long-duration Mini and R2 runs remain pending	Clean thousands/tens of thousands of banks prove basic behavior, not months of environmental exposure.

The editorial answer

The stock image did not merely leave performance on the table. It encoded optimistic assumptions as infinite loops, omitted ownership where two independent DMA engines shared memory, reset a USB protocol state at the wrong lifecycle boundary, and compiled its DMA error acknowledgement out of production. Those are defects.

Most stock users probably experienced them as an occasional click, a bad first block, a radio that needed unplugging, or simply a device that became unreliable on a shared bus. That subtlety does not exonerate the code; it explains why the defects survived. The failures were either silent or rare, and the original image had almost no telemetry with which to assign blame.

V5c is not finished engineering. It is, however, a decisive improvement in the most important sense: the firmware now knows who owns each sample bank, distinguishes delay from loss, and has bounded behavior for the transport errors most likely to turn a recoverable burp into a wedged radio.

Source and evidence index

The following local records were used to reconstruct the findings. Stock snippets are from git tag/commit 946184a unless otherwise labeled.

Record	Primary evidence
airspy_m0/airspy_m0.c	Offset handoff, blocking submission, startup ordering, cross-core waits.
airspy_m0/airspy_rx.c	Receiver lifecycle and endpoint reinitialization.
airspy_m0/usb_endpoint.c	One-entry bulk-IN dTD pool.
common/usb_queue.c	Blocking allocator, queue locking, dTD preparation, fatal error loop.
common/usb.c	Prime/flush/reset/tripwire waits, bus reset order, ignored suspend/error IRQs.
common/usb_request.c	EP0 completion ordering.
airspy_m4/airspy_m4.c	GPDMA production error defect, W1C operations, offset publication.
2026-07-23-mini-usb-hardware-qualification.md	Toggle-loss reproducer, SRAM visibility and ATDTW hardware stress.
2026-07-23-r2-10msps-live-qualification.md	Prepared-start FIFO diagnosis and V4 pointer-cache regression.
2026-07-23-long-run-longevity-audit.md	Static long-run defect inventory and remaining gaps.
2026-07-23-hardening-pass-1.md	GPDMA provenance, EP0, stale retirement, validation and build dependency.
2026-07-23-hardening-pass-2-recoverable-streaming.md	Recoverable DMA/USB state machines, deadlines and V5c SRAM-order correction.
hardware/images/README.md	Final V5c identity, hash and live qualification summary.

Final image: airspy-ring-v5c-universal-alternating-sram.bin

SHA-256: 138d64f1a3dd4007f813264f721435729b6180cd5ff0851baa36939586b888d5

Status: experimental, materially hardened, not formally hardened.